Le petit modèle entre les gros

Fil B (B.0 → B.4) : ce qui a été construit, comment ça marche, ce que ça change

Le but, en une phrase

Prendre de gros agents frontière — Claude, GPT, Gemini — et intercaler entre eux un petit modèle (Qwen3-0.6B, six cents millions de paramètres, exécuté localement) qui traduit la sortie de l'un vers une forme canonique que l'autre peut consommer sans ambiguïté. Le petit modèle ne décide rien : il traduit la forme, et il s'efface honnêtement quand il ne peut pas le faire.

Ce qui existe aujourd'hui, en cinq faits vérifiés

1. Le traducteur est SERVI : une variable d'environnement (EFE_SLM_REFORMULATOR_URL) suffit à câbler le vrai petit modèle sur le chemin servi des missions relay, vote et debate de la gateway. Sans elle, rien ne change : le chemin par défaut est strictement identique, octet pour octet.
2. Chaque passage (« hop ») est estampillé du moteur qui l'a réellement produit : SLM quand le petit modèle a traduit et que sa sortie a été validée ; RULE quand la règle déterministe a servi ; SLM_DEGRADED_TO_RULE quand le SLM était censé traduire mais a été remplacé — et depuis B.4, la raison précise de chaque dégradation voyage avec le hop.
3. Ces estampilles ne sont pas des logs : elles sont SCELLÉES dans un reçu cryptographique (chaîne HMAC) re-vérifiable hors ligne. Maquiller après coup une dégradation en succès casse la re-preuve, et la falsification est localisée.
4. Le petit modèle est protégé et borné : un circuit à sonde unique coupe les appels vers un serveur malade, et chaque hop est plafonné au temps de mission RESTANT — un traducteur pendu ne peut plus voler trente secondes à une mission qui n'en a que deux.
5. Tout cela est prouvé : 1 016 tests verts sur 45 suites, quatre revues adverses (43 findings bruts, 19 défauts distincts corrigés), et un audit final de complétude — six vérificateurs indépendants, 45 contrôles, zéro trou contre la spécification.

La règle cardinale qui a tout structuré

Ne jamais sur-vendre. La revendication « relais médié par le petit modèle » n'est vraie que hop par hop, que si le SLM a réellement tourné et réussi, et toute dégradation vers la règle est visible, datée, scellée. Ce rapport suit la même règle : chaque schéma qui suit montre aussi ce que le système ne fait PAS.

Avant / après — ce que le Fil B a changé

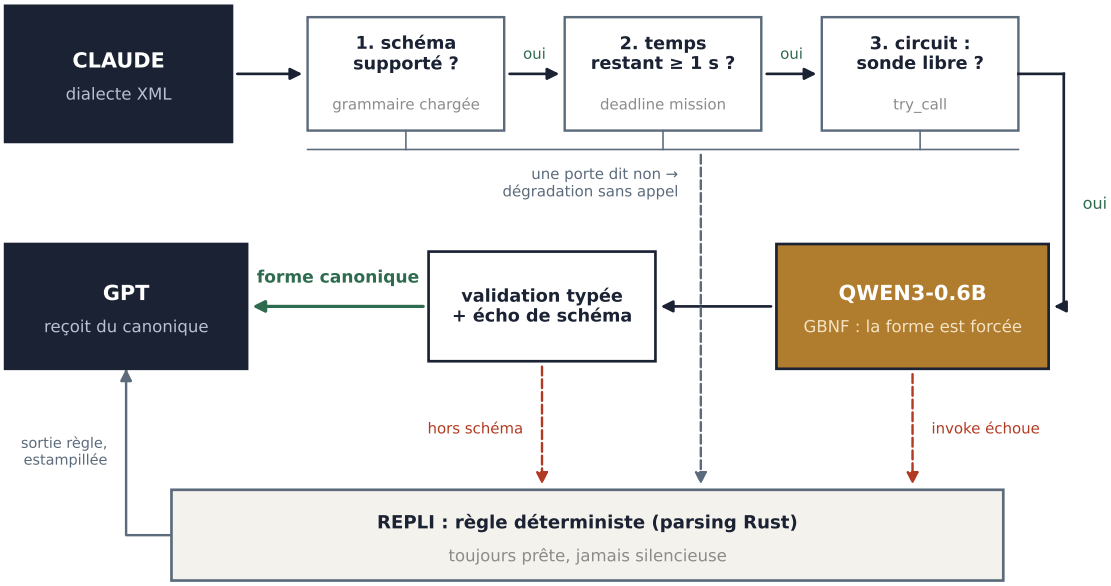
Avant le Fil B, la reformulation entre gros modèles existait déjà — mais sous une seule forme : un parseur Rust déterministe, correct mais invisible en tant que choix, et dont personne ne pouvait prouver ni l'usage ni l'absence. Le tableau ci-dessous confronte, point par point, l'état d'avant (commit e66cca2 et antérieurs) à l'état d'aujourd'hui (HEAD 02eccdb).

Aspect	AVANT (e66cca2)	APRÈS (02eccdb)
Le traducteur	Parsing Rust déterministe seul, construit en dur dans chaque mission	Vrai petit modèle (Qwen3-0.6B, GBNF) servi opt-in + repli règle estampillé
La preuve « SLM-médié »	Invérifiable : évènement tronqué à 120 caractères sur un bus sans abonné	Scellée par hop dans le reçu ; vraie ssi TOUS les sceaux sont SLM
L'audit du petit modèle	Mourait dans une mémoire locale, jeté à la fin du hop	Committé : empreintes entrée/sortie, tokens, latence — croisable par id
Le « hash » d'audit	audit_chain_hash = l'identifiant de mission reformaté (pas un hash)	Nœuds scellés dans la chaîne HMAC + racine re-vérifiable hors ligne
Panne du traducteur	Jusqu'à 30 s perdues par hop ; tempête d'appels sur serveur malade	Circuit à sonde unique + hop borné au temps de mission restant
La dégradation	Silencieuse, indiscernable d'un choix délibéré	Estampillée + motivée : six raisons stables, de deadline à circuit ouvert
La falsification	Rien à falsifier : rien n'était scellé	Maquiller un seul sceau casse la re-preuve — localisée

Lecture honnête du tableau : la colonne « après » ne dit pas que le petit modèle traduit toujours, ni qu'il traduit bien — elle dit que ce qu'il fait (ou ne fait pas) est désormais mesurable, scellé et re-vérifiable. La qualité de traduction en conditions réelles reste vérifiée par des tests live qui exigent un llama-server local (marqués #[ignore] hors CI) — c'est une réserve documentée, pas un oubli.

Schéma 1 — le fonctionnement, vu de haut

Un « hop » de reformulation est le trajet d'un texte entre deux gros modèles. Le schéma suit ce trajet de gauche à droite : la sortie brute de l'émetteur traverse trois gardes, puis le petit modèle (ou son repli), et ressort sous forme canonique. Tout en bas, la piste d'audit descend vers le reçu scellé.



Chaque sortie porte son estampille :

SLM	le petit modèle a traduit, sortie validée
RULE	la règle a servi (aucun SLM câblé)
SLM_DEGRADED_TO_RULE	le SLM devait traduire, la règle a remplacé — raison scellée

Et pendant ce temps, en dessous — la piste d'audit :



Trois points méritent l'attention. D'abord, l'ordre des gardes n'est pas décoratif : la deadline se vérifie AVANT le circuit, pour qu'un hop sans temps ne consomme pas la précieuse sonde de rétablissement. Ensuite, la validation typée ne fait pas confiance à la grammaire : même contrainte par GBNF, la sortie du petit modèle est re-désérialisée dans la forme canonique attendue, et l'écho du schéma est vérifié — défense en profondeur. Enfin, la piste d'audit fonctionne aussi pour les tentatives ÉCHOUÉES : un invoke qui meurt en vol laisse quand même son empreinte dans le reçu.

Schéma 2 — l'arc du chantier, B.0 → B.4

Le Fil B a été livré en cinq jalons, chacun commité, testé et passé au feu d'une revue adverse (des agents indépendants chargés de réfuter le travail). L'ordre n'est pas arbitraire : chaque jalon rend le suivant possible et sûr.



B.0 a posé la couture : un trait Rust « Reformulator » avec deux implémentations interchangeables — la règle déterministe (l'existant) et l'adaptateur du petit modèle (dormant au départ). L'invariant fondateur, prouvé par test : tant qu'aucun SLM n'est câblé, la sortie des missions est identique à l'octet près. On pouvait donc merger sans risque.

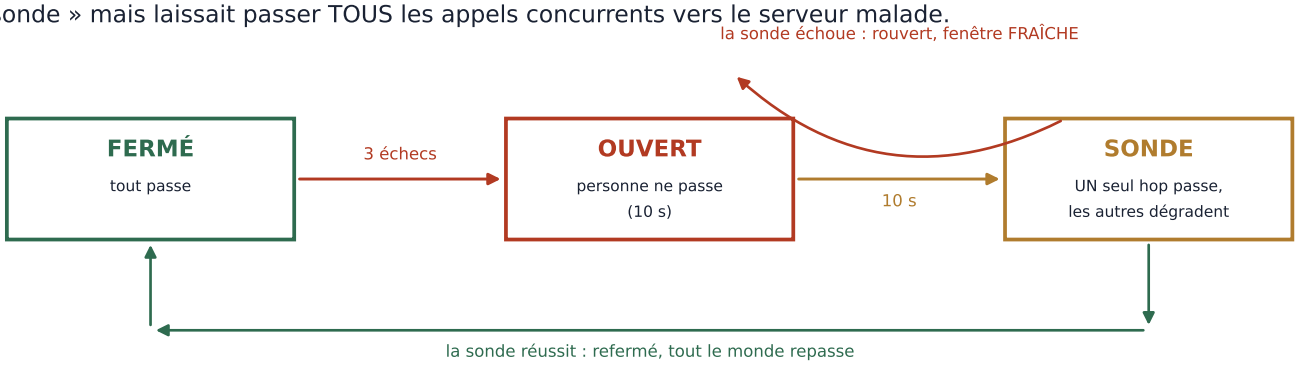
B.1 a donné la substance : le petit modèle ciblait un vocabulaire de schémas différent de celui des missions (citation, fact, tool-call… contre plan, code, claim, argument, synthèse). Il a fallu de nouveaux actifs — une grammaire GBNF en union de cinq enveloppes qui LIE chaque identifiant de schéma à sa forme exacte (le mensonge de forme devient impossible par construction), un prompt à few-shots, et un schéma JSON dont les exemples servent de contrat testé entre les crates.

B.2 a branché le tout sur le chemin servi — derrière une variable d'environnement explicite, avec un boot qui ne casse JAMAIS (assets absents = avertissement + règle) — et a tiré de B.4 le circuit breaker, parce qu'un circuit enregistré mais non consommé aurait été une protection de façade. La revue adverse y a durci l'infrastructure partagée : porte à sonde unique, fenêtres de réouverture sans course.

B.3 a rendu la promesse de provenance réelle : les estampilles n'allaient nulle part (bus interne sans abonné, payload tronqué). Désormais chaque hop est un nœud du graphe scellé de mission, committé individuellement dans la chaîne HMAC. B.4 a fermé le dernier trou : le temps. Un hop est plafonné au reste de mission ; couper un invoke en vol synthétise son empreinte d'audit ; et l'impatience d'une mission ne compte pas comme une maladie du robot.

Schéma 3 — le circuit à sonde unique (B.2)

Le circuit breaker répond à une question simple : faut-il encore appeler un llama-server qui ne répond plus ? Trois échecs consécutifs ferment la porte pendant dix secondes. Ensuite commence la partie subtile — la période de sonde — que la revue adverse de B.2 a durcie : la version d'origine promettait « une sonde » mais laissait passer TOUS les appels concurrents vers le serveur malade.



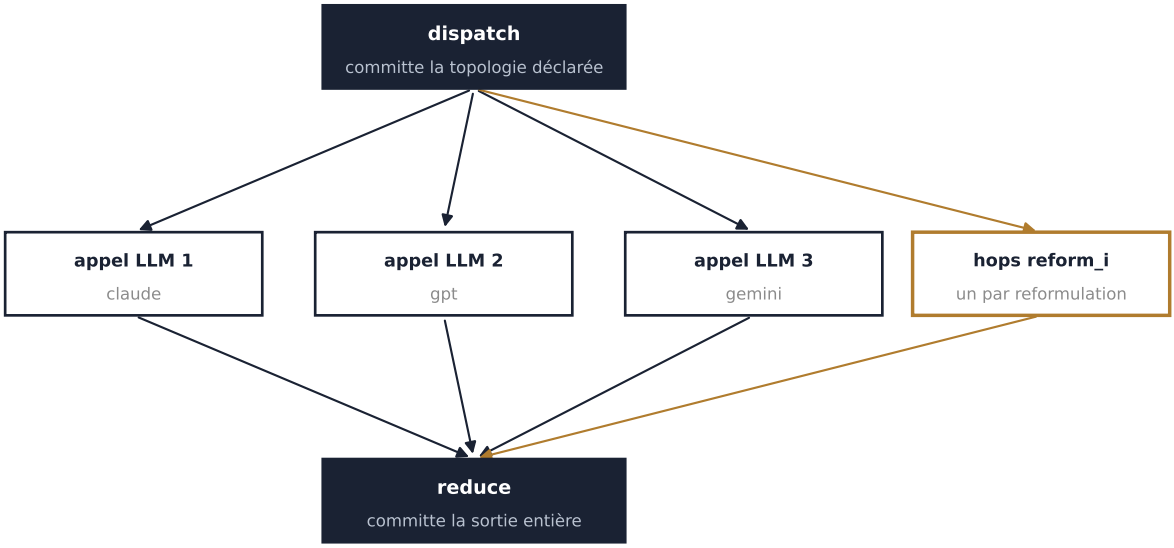
Les trois garanties durcies par la revue : (1) UNE sonde à la fois — le premier hop qui se présente acquiert le droit d'essayer, les concurrents dégradent sans appel réseau, donc un serveur en train de redémarrer reçoit exactement une requête et pas un troupeau ; (2) une sonde qui ne rend jamais de verdict (appelant coupé) libère son créneau après la fenêtre de reprise — pas de blocage éternel ; (3) la réouverture écrit son horodatage AVANT de publier l'état, pour qu'un observateur concurrent ne puisse pas re-basculer une porte fraîchement fermée.

Ce que le circuit ne mesure PAS (et c'est voulu)

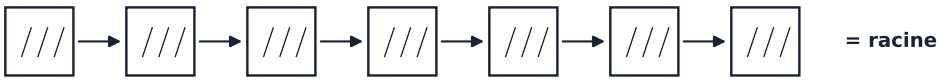
Le circuit mesure la santé du ROBOT contre son propre contrat de service (trente secondes), pas la qualité des entrées ni l'impatience des missions. Une entrée vide (rejet côté client) ne compte pas. Une coupure par deadline de mission (B.4) ne compte pas non plus : trois missions pressées ne doivent pas bloquer dix secondes un robot parfaitement sain. Seuls comptent les vrais échecs du robot : transport, timeout de trente secondes, sortie hors schéma, écho faux. Réserve honnête, documentée : une génération abandonnée peut continuer de moudre CÔTÉ serveur et faire payer aux hops suivants de vrais timeouts — un impact indirect que le tag direct ne masque pas.

Schéma 4 — le reçu scellé (B.3)

À la fin d'une mission, la gateway construit un graphe de provenance depuis la topologie DÉCLARÉE (jamais depuis ce que le producteur raconte), y ajoute un nœud par hop de reformulation, puis scelle le tout dans une chaîne HMAC. Le schéma montre le graphe d'un vote à trois modèles.



puis tout est scellé, maillon par maillon :



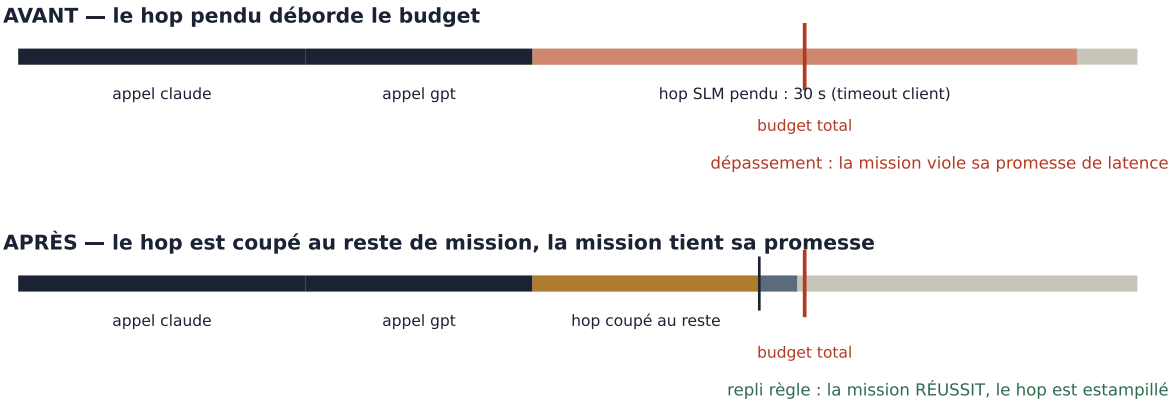
chaque maillon = HMAC(clé du tenant, maillon précédent + contenu du nœud)

Trois propriétés font du reçu autre chose qu'un log. L'INJECTIVITÉ : chaque enregistrement de provenance ne peut être consommé que par un seul nœud déclaré — impossible de faire passer un appel pour trois. La DÉCLARATION : l'arité vient de la requête (trois modèles demandés = trois nœuds), donc un appel omis crée un nœud « manquant » et la re-preuve échoue. Le SCELLEMENT INDIVIDUEL des hops : le JSON complet de chaque hop — moteur, raison de dégradation, empreintes d'audit du petit modèle — est committé dans son propre nœud ; maquiller après coup un SLM_DEGRADED_TO_RULE en SLM casse la chaîne à cet endroit précis.

La surface servie expose le verdict en clair : la réponse HTTP de mission porte un bloc « reformulation » avec le compte des hops par moteur et le drapeau slm_mediated — vrai si et seulement si TOUS les hops sont SLM. Et un vérificateur hors ligne peut re-prouver le reçu sans faire confiance au serveur : la clé du tenant suffit. Réserve honnête : le détenteur de la clé reste l'autorité — c'est du tamper-evident, pas du trustless vis-à-vis d'un tiers.

Schéma 5 — le temps borné (B.4)

Chaque mission part avec un budget de temps total (max_total_latency). Avant B.4, les appels aux gros modèles étaient bien plafonnés au temps restant, mais le hop de reformulation ne l'était pas : un petit modèle pendu pouvait coûter trente secondes — le timeout de son client — à une mission qui n'avait plus que deux secondes de budget. Le schéma montre la même mission, avant et après.

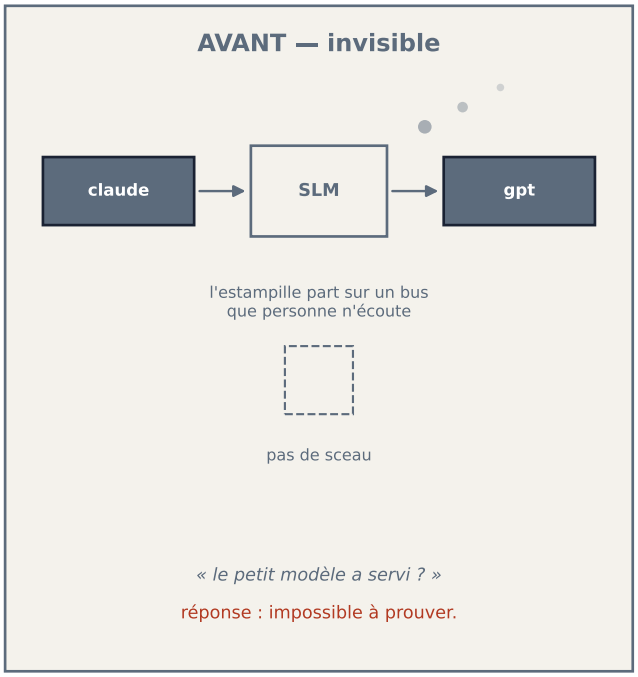


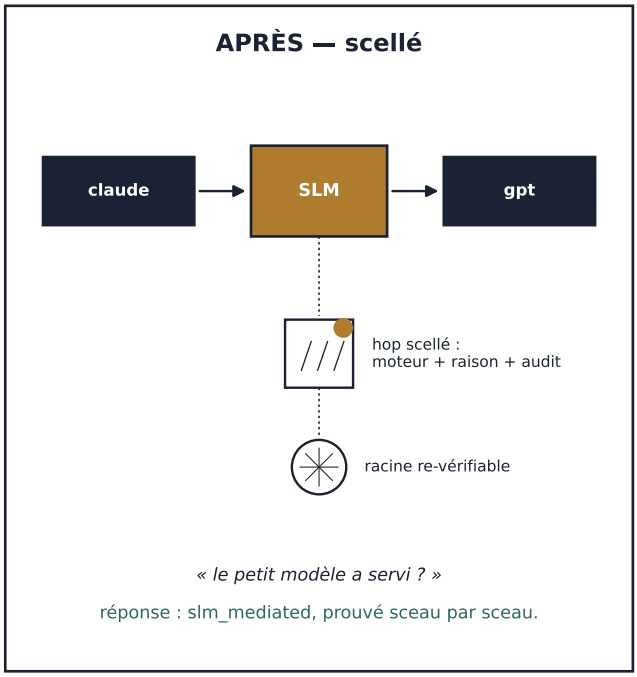
Les quatre décisions qui rendent la coupe honnête

- Le plancher d'une seconde. Sous une seconde de reste, on ne tente même pas : un vrai Qwen3-0.6B ne répond pas si vite, et l'essai ne ferait que brûler le budget de l'appel frontière suivant. Dégradation immédiate, sans appel réseau, sans consommer la sonde du circuit.
- La coupe synthétise l'empreinte. Couper un invoke en vol annule aussi l'écriture de son audit. Le backend synthétise alors l'entrée manquante (statut TIMEOUT, même empreinte d'entrée que le client aurait committée) — l'invariant de B.3, « tentative réelle = tentative capturée », tient sous B.4.
- L'impatience n'est pas une maladie. Une coupure-deadline ne compte pas au circuit : le circuit mesure le robot contre son contrat de trente secondes, pas contre l'urgence des missions. Sinon trois missions pressées suffiraient à bloquer un robot sain.
- La raison voyage avec le hop. Chaque dégradation porte un tag stable — deadline_exhausted, circuit_open, robot_error, invalid_output, unsupported_schema, client_side — scellé dans le reçu. Une deadline épuisée ne ressemble plus à un circuit ouvert.

Schéma 6 — le même hop, deux mondes

Pour finir, la comparaison la plus parlante : le trajet d'un même passage, avant et après le Fil B. À gauche, tout fonctionne — mais rien n'est prouvable. À droite, le même trajet laisse une piste scellée que n'importe quel auditeur muni de la clé peut re-vérifier hors ligne.





Les chiffres de la traversée

6	commits, de la couture à l'audit final (04759dd → 02eccdb)
1 016	tests verts sur 45 suites au dernier passage — zéro échec
43 → 19	findings bruts des quatre revues adverses → défauts distincts corrigés
45 / 0	contrôles de l'audit de complétude / trous découverts

Et les réserves, dites une dernière fois parce qu'elles font partie du travail : la qualité de traduction en conditions réelles n'est vérifiée que par les tests live (llama-server requis, hors CI) ; les poids du modèle ne sont pas embarqués ; l'URL du serveur n'est pas contrainte au local (le boot avertit) ; et une double défaillance — petit modèle ET règle — perd la tentative du reçu, qui reste observable dans les journaux et le writer partagé. Un système qui énonce ses limites est exactement ce que le reçu promet : de l'honnêteté re-vérifiable.